Experiment 6

Student Name: Lakshay Aggarwal
UID: 25MCI10047
Branch: MCA (AI/ML)
Section/Group: 25MAM-1(A)
Semester: 2-SEM
Date of Performance: 03/03/2026
Subject Name: Technical Training - 1
Subject Code: 25CAP-652

1. Aim of the Session:

Learn how to create, query, and manage views in SQL to simplify database queries and provide a layer of abstraction for end-users.

2. Software Requirements:

- PostgreSQL
- pgAdmin
- Windows Operating System

3. Objectives of the Session:

- **Data Abstraction:** To understand how to hide complex table joins and calculations behind a simple virtual table interface.
- **Enhanced Security:** To learn how to restrict user access to sensitive columns by providing views instead of direct table access.
- **Query Simplification:** To master the creation of views that pre-join multiple tables, making reporting easier for non-technical users.
- **View Management:** To understand the syntax for creating, altering, and dropping views, as well as the naming conventions required for efficient data access.

4. Theory:

A **View** is essentially a virtual table based on the result-set of an SQL statement. It does not contain data of its own but dynamically pulls data from the underlying "base tables".

1. **Simple Views:** Created from a single table without any aggregate functions or grouping. These are often updatable.
2. **Complex Views:** Created from multiple tables using JOINs, or including GROUP BY and aggregate functions. These provide a consolidated summary of the database.

3. **Security Layer:** In enterprise environments, views are used to grant permissions on specific subsets of data. For example, a "SalaryView" might exclude the "Employee_SSN" or "Home_Address" columns for privacy.
4. **Benefits:** They simplify the user experience, ensure data consistency across reports, and reduce the risk of accidental data modification by providing read-only abstractions.

5. Practical / Experiment Steps:

Step 0: Creating sample tables and inserting records

Code:

```
CREATE TABLE departments (
   dept_id SERIAL PRIMARY KEY,
   dept_name VARCHAR(50)
);

CREATE TABLE employees (
   emp_id SERIAL PRIMARY KEY,
   emp_name VARCHAR(50),
   status VARCHAR(10),
   salary INT,
   dept_id INT REFERENCES departments(dept_id)
);

INSERT INTO departments (dept_name) VALUES
('HR'),
('IT'),
('Marketing'),
('Sales');

INSERT INTO employees (emp_name, status, salary, dept_id) VALUES
('Lakshay', 'ACTIVE', 50000, 2),
('Neha', 'ACTIVE', 60000, 3),
('Tushar', 'INACTIVE', 40000, 2),
('Priya', 'ACTIVE', 70000, 4),
('Tanu', 'ACTIVE', 45000, 1),
('Palak', 'INACTIVE', 30000, 1);

SELECT * FROM departments;
SELECT * FROM employees;
```

Output:

	dept_id [PK] integer	dept_name character varying (50)
1	1	HR
2	2	IT
3	3	Marketing
4	4	Sales

	emp_id [PK] integer	emp_name character varying (50)	status character varying (10)	salary integer	dept_id integer
1	1	Lakshay	ACTIVE	50000	2
2	2	Neha	ACTIVE	60000	3
3	3	Tushar	INACTIVE	40000	2
4	4	Priya	ACTIVE	70000	4
5	5	Tanu	ACTIVE	45000	1
6	6	Palak	INACTIVE	30000	1

Step 1: Creating a Simple View for Data Filtering

Implementing a view to provide a quick list of active employees without exposing the entire table structure.

Code:

```
CREATE VIEW active_employees AS
SELECT emp_id, emp_name, salary
FROM employees
WHERE status = 'ACTIVE';

SELECT * FROM active_employees;
```

Output:

	emp_id integer	emp_name character varying (50)	salary integer
1	1	Lakshay	50000
2	2	Neha	60000
3	4	Priya	70000
4	5	Tanu	45000

Step 2: Creating a View for Joining Multiple Tables

Simplifying the retrieval of data distributed across Employees and Departments tables.

Code:

```
CREATE VIEW employee_department_view AS
SELECT
   e.emp_id,
   e.emp_name,
   d.dept_name,
   e.salary
FROM employees e
JOIN departments d
ON e.dept_id = d.dept_id;

SELECT * FROM employee_department_view;
```

Output:

	emp_id integer	emp_name character varying (50)	dept_name character varying (50)	salary integer
1	1	Lakshay	IT	50000
2	2	Neha	Marketing	60000
3	3	Tushar	IT	40000
4	4	Priya	Sales	70000
5	5	Tanu	HR	45000
6	6	Palak	HR	30000

Step 3: Advanced Summarization View

Creating a view to provide department-level statistics automatically.

Code:

```
CREATE VIEW department_statistics AS
SELECT
   d.dept_name,
   COUNT(e.emp_id) AS total_employees,
   AVG(e.salary) AS avg_salary,
   MAX(e.salary) AS max_salary
FROM departments d
LEFT JOIN employees e
ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

SELECT * FROM department_statistics;

Output:

	dept_name character varying (50)	total_employees bigint	avg_salary numeric	max_salary integer
1	Marketing	1	60000.000000000000	60000
2	Sales	1	70000.000000000000	70000
3	IT	2	45000.000000000000	50000
4	HR	2	37500.000000000000	45000

6. Learning Outcomes (What I have learned):

- **Abstraction Proficiency:** Students will be able to create and query views to simplify efficient data access and abstraction.
- **Security Implementation:** Students will understand how to use views for data masking and providing restricted access to sensitive information.
- **Syntactic Accuracy:** Students will demonstrate the correct syntax for creating and querying views, ensuring logical clarity in naming conventions.
- **Real-world Application:** Students will be able to design views for practical domains like Library Management Systems or Payroll Systems to demonstrate functionality.